

Long-Running 编码智能体系统

一、问题建模与方法设计

1.1 问题定义

给定一个自然语言描述的软件工程任务 T （例如“实现一个 HTML 清理库”），目标是在给定的资源预算（会话数 S_{max} 、运行时间 H_{max} ）内，由 LLM 驱动的编码智能体自主完成从任务分解、代码编写、测试验证到最终交付的全流程，无需人工介入。

形式化地，任务 T 被 Planner 分解为一个特征集合 $\mathcal{F} = \{f_1, f_2, \dots, f_n\}$ ，每个特征 f_i 包含：

- 自然语言描述 d_i
- 自动化验收标准 $a_i = (cmd_i, expect_exit_i)$
- 依赖关系 $dep_i \subseteq \mathcal{F} \setminus \{f_i\}$
- 状态 $s_i \in \{pending, in_progress, passing, failed, blocked, skipped\}$

系统的目标是在预算约束下最大化最终通过验证的特征数量：

$$\begin{aligned} \max \quad & |\{f_i \in \mathcal{F} : s_i = passing\}| \\ \text{s.t.} \quad & sessions \leq S_{max}, \quad wall_clock \leq H_{max} \end{aligned}$$

1.2 核心挑战

LLM 驱动的自主编程面临三个关键瓶颈：

(1) 单次上下文容量有限。 当前主流 LLM 的上下文窗口（DeepSeek V4 为 1M tokens）虽大，但在复杂软件工程任务中仍不够，一个中等规模的项目可能涉及数十个文件、上百次编译运行、数千行输出。当上下文被历史对话占满时，模型无法继续有效工作。

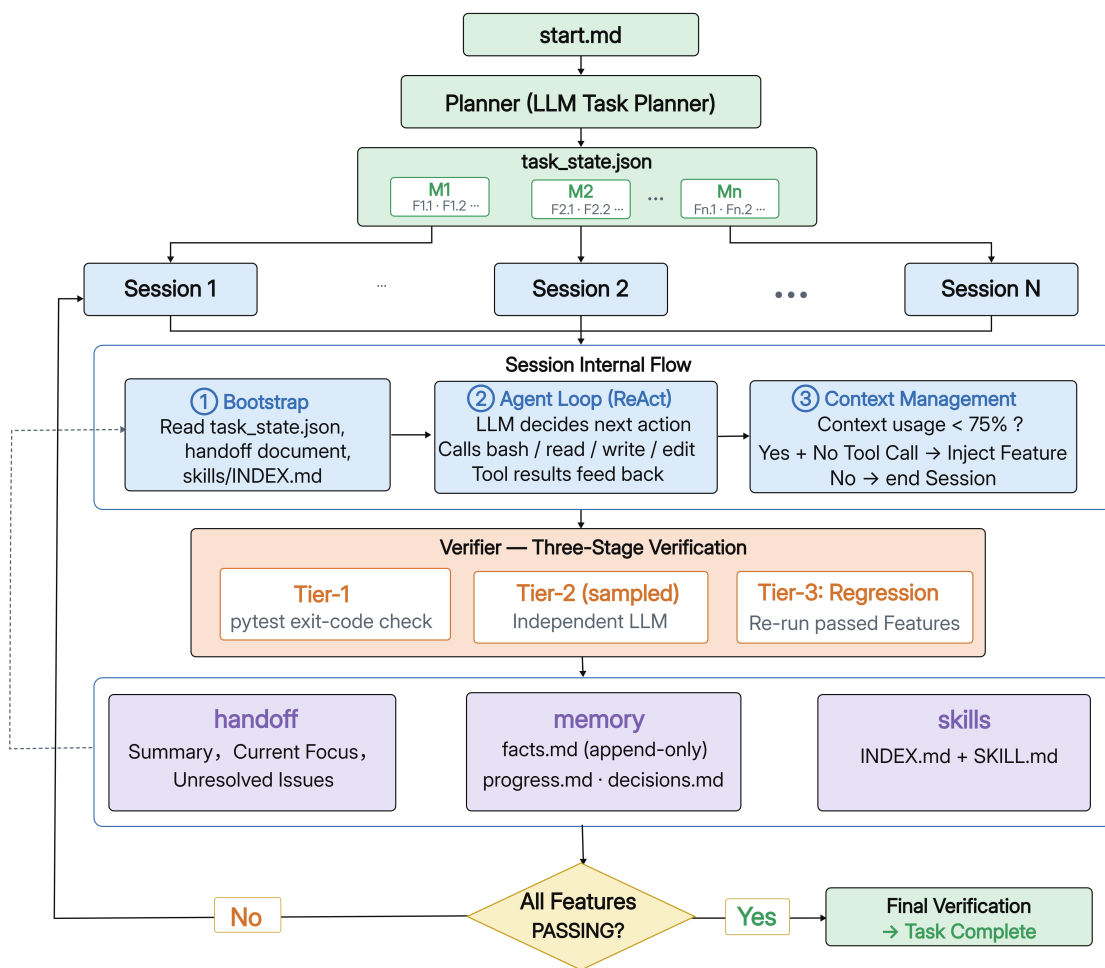
(2) 模型存在“幻觉完成”倾向。 LLM 倾向于声称任务已完成，即使代码存在编译错误或逻辑缺陷。若不加外部验证机制，智能体可能在未真正实现功能时就停止工作，造成“假完成”。

(3) 经验无法跨会话复用。 每次新会话都是一个“失忆”的智能体——它不记得之前的失败尝试、已发现的 API 约束、或已验证有效的修复模式。这在长时间运行中导致大量重复试错。

1.3 方法设计

针对上述挑战，本系统设计了三层架构：**编排层 (Orchestrator)** 负责会话生命周期管理和全局决策；**执行层 (Agent Loop)** 在单会话内运行标准 ReAct 循环；**验证层 (Verifier)** 提供独立于生成过程的三阶段质量门禁。三层通过文件系统上的结构化状态（`task_state.json`）和追加式记忆文件（`progress.md`、`decisions.md`、`facts.md`）实现解耦通信。

1.4 方法示意图



本系统采用三层闭环架构。任务规范（start.md）首先送入 Planner 规划器，由 LLM 将其分解为结构化的 Milestone-Feature 树并写入 task_state.json 作为全局唯一真相源。随后系统进入多 Session 执行循环：每个新会话启动时通过 Bootstrap 阶段从 task_state.json、handoff 交接文档和 skills 技能索引中恢复上下文，然后在 Agent Loop 中以 ReAct 模式反复进行“LLM 决策→工具调用（bash/read/write/edit）→结果反馈”的编码循环；当上下文使用率达到 75% 阈值时，会话结束并触发 Verifier 双阶段验证：先通过 Tier-1 自动执行 pytest 验收命令检查退出码，再以 30% 采样率启动独立的 Tier-2 LLM 会话进行盲审（已失败过的 Feature 则 100% 审查），最后对已通过 Feature 做回归检测防止新代码破坏旧功能。验证结果通过后，编排器自动生成结构化 handoff 交接文档、更新 memory 记忆文件（progress/decisions/facts），并从多次重试后成功的 Feature 中自动提取 Skill 修复模式存入技能库。这些持久化内容通过回路反馈给下一 Session 的 Bootstrap 阶段，形成跨会话的知识传递闭环；系统在全部 Feature 均通过或跳过后执行最终全量验证，任务完成。

二、实验设置

2.1 基准测试集

实验采用 NL2RepoBench 基准测试集，该数据集包含 100+ 个 Python 开源项目任务。每个任务提供：

- `start.md`：任务规范文档，描述目标库的核心功能、API 签名和伪代码
- `test_commands.json`：评估阶段的 pytest 测试命令
- `test_files.json`：测试时需要从工作区排除的测试文件列表，防止评估沙箱中的测试文件被覆盖
- `test_case_count.txt`：预期测试用例总数，用于计算得分

每个任务配套一个 Docker 测试镜像（`ghcr.nju.edu.cn/multimodal-art-projection/nl2repobench/<task>:1.0`），其中预装了测试依赖和参考答案。评估时，将智能体编写的代码叠加到测试镜像上运行 `pytest`，通过率即为得分。

2.2 运行环境

组件	规格
LLM 模型	DeepSeek V4 Pro（1.6T/49B 激活参数，1M 上下文窗口）
Planner 模型	DeepSeek V4 Pro（thinking 模式，temperature=0.3）
Verifier 模型	DeepSeek V4 Pro（thinking 模式，temperature=0.2）
Python 版本	3.11
运行沙箱	Docker 容器 <code>ghcr.nju.edu.cn/all-hands-ai/runtime:0.56-nikolaik</code>
评估沙箱	Docker 容器 <code>ghcr.nju.edu.cn/multimodal-art-projection/nl2repobench/<task>:1.0</code>

2.3 智能体配置

参数	值	说明
<code>max_turns</code>	200	每会话最大工具调用轮数
<code>max_tokens</code>	750,000	每会话 token 预算（上下文窗口的 75%）
<code>context_window</code>	1,000,000	模型上下文窗口大小
<code>switch_ratio</code>	0.75	上下文使用率达到该比例时切换会话
<code>max_sessions</code>	40	总会话数上限
<code>max_wall_clock_hours</code>	8	总运行时间上限
<code>tier2_sample_rate</code>	0.3	Tier-2 LLM 审查的采样率
<code>max_retry_attempts</code>	5	Feature 最大重试次数
<code>replan_max_count</code>	3	最大重规划次数

2.4 评估指标

- Score:** `passed_tests/expected_test_count`（上限 1.0），即 `pytest` 通过率
- Status:** `passed`（全部通过） / `failed`（有失败用例） / `error`（执行异常）
- 辅助指标:** 总会话数、总运行时间、Feature 完成率、验证通过率、Tier-2 审查结果

三、系统组成

3.1 基础智能体循环

基础智能体循环采用标准的 **ReAct (Reasoning + Acting)** 模式。每个会话内，系统维护一个消息列表 `messages = [system_prompt, bootstrap..., assistant_1, tool_result_1, assistant_2, ...]`，循环执行以下步骤：

- 1. **LLM 推理**：将当前消息列表发送给 LLM，LLM 返回文本响应和/或工具调用
- 2. **工具执行**：若 LLM 请求工具调用，系统在 Docker 沙箱中执行对应操作并返回结果
- 3. **预算检查**：检查 turns 和 tokens 是否超出会话预算
- 4. **空闲处理**：若 LLM 停止且无工具调用（`end_turn`），触发 `on_idle` 回调，由编排器判断是注入下一个 Feature 还是结束会话

工具集设计保持最小化，仅包含四个操作：

工具	功能	安全约束
<code>bash(command, timeout_sec)</code>	在 Docker 容器中执行 shell 命令	禁止 <code>cd ..</code> 越权；stdout/stderr 截断（8000 字符）；超时控制（120s）
<code>read(path, start_line, end_line)</code>	读取文件内容，支持行范围	仅允许 <code>workspace/</code> 和 <code>.agent/</code> 路径；大文件截断（16000 字符）
<code>write(path, content, mode)</code>	写入或追加文件	<code>.agent/memory/*.md</code> 强制 append-only；仅允许路径白名单
<code>edit(path, old_string, new_string)</code>	精确字符串替换	旧串必须唯一匹配（或 <code>replace_all=true</code> ）；仅允许白名单路径

工具集的最小化设计基于以下考虑：（1）减少 LLM 的工具选择困惑；（2）降低安全风险面；（3）`edit` 工具使 LLM 可以进行 O(1) 的精准修改，避免“读整文件→改→写整文件”的低效模式。

3.2 长时程运行能力

3.2.1 任务状态管理

任务分解。 系统使用独立的 Planner 智能体将任务规范 `start.md` 分解为结构化的 Milestone-Feature 树。Planner 的输出是一个 JSON 对象，包含：

```
{
  "task_id": "decouple",
  "milestones": [
    {
      "id": "M1",
      "title": "Core Configuration Classes",
      "features": [
        {
          "id": "F1.1",
          "description": "Implement config base class with __call__ casting",
          "depends_on": [],
          "acceptance": {
```

```

        "type": "automated_test",
        "command": "cd workspace && python -m pytest
tests/test_config_core.py -q",
        "expect_exit_code": 0
    },
    "status": "pending",
    "attempts": 0
}
],
"final_verification": {
    "command": "cd workspace && python -m pytest tests/ -q --tb=short",
    "status": "pending"
}
}

```

规划粒度。 Planner 遵循"一个 Feature = 一个可端到端演示和自动验证的最小行为单元"原则。粒度选择遵循：不应小到"改一行代码"，也不应大到"实现整个模块"。如果模块特别复杂，拆分为更多子 Feature。每个 Feature 必须包含明确的、可执行的 acceptance 命令。

工作选择算法。 编排器通过 `pick_next_feature()` 函数选择下一个要执行的 Feature，优先级为：

1. **失败重试：** 状态为 `failed` 且重试次数 $\leq$ `max_retry_attempts` 的 Feature（优先处理可修复的问题）
2. **依赖就绪：** 状态为 `pending` 且所有依赖已 `passing` 的 Feature
3. **阻塞恢复：** 状态为 `blocked` 但现在依赖已满足的 Feature

终止条件。 系统在所有 Feature 均为 `passing` 或 `skipped` 且最终验证通过时终止，或在达到会话数/时间预算上限时终止。

状态机约束。 Feature 状态转换受严格规则限制：

- 待处理 (`pending`)：可以转换为进行中，或转换为跳过。
- 进行中 (`in_progress`)：可以转换为已验证通过、失败、阻塞，或回退为待处理。
- 已验证通过 (`passing`)：只能由验证器降级为失败。这通常发生在回归检测中，即后续修改破坏了已通过的 Feature。智能体自身无权将通过的 Feature 标为其他状态。
- 失败 (`failed`)：可以转换为进行中（重新尝试）、待处理（等待下次调度），或跳过（不再需要）。
- 阻塞 (`blocked`)：当阻塞原因消除后，可以转换为待处理、进行中，或跳过。
- 跳过 (`skipped`)：可以恢复为待处理，用于应对情况变化后重新需要该 Feature 的场景。

关键约束： `passing` $\rightarrow$ `failed` 只允许通过 verifier 的回归检测触发，智能体自身不能将已通过的 Feature 标记为失败。这防止模型在修改其他 Feature 时"不小心"破坏已验证的工作。

动态重规划。 当满足以下任一条件时，编排器触发 Planner 重新分解任务：

- 连续验证失败次数 $\geq$ `replan_consecutive_failures`（默认 5）
- 所有剩余 Feature 均为 `blocked` 状态
- 模型在会话中显式请求 `replan`

重规划时，已 `passing` 的 Feature 被保留，移除的旧 Feature 标记为 `skipped`，新增 Feature 追加到对应 Milestone。存活下来的非 `passing` Feature 的 `attempts` 计数被重置——因为 Planner 重新设计了方案，旧方法的失败次数不应惩罚新方法。

3.2.2 上下文管理

系统采用上下文驱动的会话切换策略，而非固定的 1-Feature-per-Session 模式。

会话内压缩。 当累计 token 使用量超过上下文窗口的 50% 时，`compress_messages()` 函数触发过期工具结果清理：超过 K 轮的工具输出被替换为占位文本（`[Cleared: tool result from turn X, ~Y tokens. Re-read relevant files...]`）。这种策略比摘要式压缩更安全——不会丢失任何事实信息，只是督促模型在需要时重新读取文件。

会话切换。 当 token 使用量达到上下文窗口的 75%（`switch_ratio`）时，编排器强制结束当前会话并启动新会话。一个会话可能处理多个 Feature，当模型空闲时，`on_idle` 回调注入下一个 Feature 的 prompt 使会话继续工作。这减少了会话切换次数。

交接文档 (Handoff)。 每会话结束后，编排器自动生成结构化的交接文档（`handoffs/session_NNNN.md`），包含：

字段	内容	生成方式
Session Summary	3-5 句自然语言摘要	Planner/Flash LLM 读取截断的转录
Current Focus	当前 Feature ID 与描述	编排器记录
Last Git Commit	最后提交的 hash 和 message	<code>git log -1</code>
Unresolved Issues	未解决的错误（文件:症状）	自动扫描转录中的失败工具调用
Next Action	下一个具体操作	编排器根据任务状态推断
Do Not Do	应避免的操作	从失败 Feature 的 notes 提取
Files to Re-read	新会话应重新读取的文件路径	<code>git diff --name-only</code>

Bootstrap 序列。 每个新会话启动时，系统执行固定的引导流程：

1. 读取 `task_state.json` — 了解当前 Feature 状态和依赖关系
2. 读取最新 `handoff` 文档 — 理解上一会话的上下文
3. 读取原始 `start.md` — Planner 的摘要可能丢失细节，原始规范具有最高优先级
4. 扫描 `skills/INDEX.md` — 发现可用的跨任务经验
5. 读取 `decisions.md` 最近条目 — 了解架构约束
6. 读取 `facts.md` 最近条目 — 了解已发现的 API 行为

3.2.3 自我验证

验证层是系统中最关键的安全网，智能体不能自行标记 Feature 为"完成"，只有验证器可以授予 `passing` 状态。验证层包含三个检测阶段：

- **Tier-1: 自动验收测试。** 对每个 Feature 执行其 `acceptance.command`（通常为 `pytest`），检查 `exit_code == expect_exit_code`。这是确定性的、低成本的检查。额外保护措施：（1）系统拒绝执行看起来过于简单的命令（如 `echo "pass"` 或 `true`）；（2）若 Feature 定义了 `api_check`，同步检查公共 API 契约（导入存在性、方法签名）。

- **Tier-2: 独立 LLM 代码审查。** 以 30% 采样率（失败过的 Feature 100% 审查）启动一个独立的 LLM 会话进行深度审查。关键设计决策：Verifier 看不到编码智能体的对话历史，它只能看到：
（1）Feature 描述和验收标准；（2）当前文件内容（通过自己的 read/bash 工具）；（3）`decisions.md` 中的约束记录。这种“盲审”设计消除了编码历史中的偏见，Verifier 不会被模型的“解释”和“理由”所说服，只能基于代码事实做出判断。Verifier 必须以 `VERDICT: pass|fail` 和 `EVIDENCE: - {file:line}: {finding}` 格式输出，不允许模糊判断。
- **回归检测。** 当新 Feature 通过验证后，系统自动重跑所有之前已 `passing` 的 Feature 的 `acceptance` 命令。如果任何一个之前通过的 Feature 现在失败，说明新代码破坏了已有功能，系统同时标记新 Feature 和回归 Feature 为 `failed`。

Tier-2 的收益与代价分析：

Tier-2 的主要收益在于它能够发现 Tier-1 自动化测试漏掉的缺陷：Verifier 独立审查代码逻辑，不受测试覆盖率的限制，即使所有 pytest 用例全部通过，Verifier 仍然可能发现 API 契约不匹配、边界条件遗漏或逻辑错误。此外，Verifier 能够对照 `decisions.md` 中记录的架构约束检查代码合规性，这是纯自动化测试无法做到的。从架构层面看，将“编码”与“验证”分离到两个独立的 LLM 会话中，使编码智能体专注于实现、验证器专注于审查，避免了同一模型既当运动员又当裁判的偏见。

代价方面，最直接的是每次 Tier-2 审查消耗额外的 LLM API 调用，尽管 30% 的采样率控制了成本，但对于大规模任务集而言这仍是不可忽略的开销。采样策略本身也是一个权衡：未被采样到的 Feature 仅经过 Tier-1 验收，可能隐藏了自动化测试未能覆盖的问题。此外，Verifier 自身的判断并非绝对可靠，它也可能漏报或误报，其审查质量受限于模型能力和提示词设计。最后，双模型（编码 + 验证）架构增加了系统的整体复杂度，包括额外的会话管理、转录日志和状态同步逻辑。

3.3 Skill 与 Memory 机制

3.3.1 Skill 机制

Skill 是跨任务可复用的经验片段，以 Markdown 文件的形式存储在 `.agent/skills/<name>/SKILL.md`。系统通过两种途径生成 Skill：

(1) 编排器自动提取。 当一个 Feature 经过 ≥ 3 次尝试（`skill_extraction_min_attempts`）后最终达到 `passing` 状态时，编排器调用 Planner/Flash LLM 分析该 Feature 的会话转录，提取：

- 什么出错了（root cause）
- 修复方法是什么（fix pattern）
- 关键教训（key insight）

提取结果写入 Skill 文件，标记为 `llm-extracted` 类型。触发条件的设计意图是：需要多次尝试才成功意味着存在非显而易见的坑，值得记录为 Skill。

(2) 智能体主动创建。 编码智能体也可以在会话中通过 `write` 工具直接创建 Skill 文件（写入 `.agent/skills/<name>/SKILL.md`）。系统提示词中提供了 Skill 模板。

技能索引与发现。 编排器在每会话结束后调用 `rebuild_index()` 扫描 `.agent/skills/` 目录，自动重建 `INDEX.md`。索引以表格形式列出所有 Skill 的名称、摘要、标签、成功次数和状态。新会话启动时，Bootstrap 序列读取索引（非完整 Skill 内容），根据 Feature 的标签按需加载相关 Skill 全文。

技能生命周期。 每个 Skill 维护 `success_count` 和 `failure_count` 计数。当连续失败 ≥ 3 次且成功为 0 时，Skill 被标记为 `deprecated`。定期维护（每 10 个会话）运行去重和废弃检查。

3.3.2 Memory 机制

系统以文件系统为载体实现跨会话的状态持久化。Memory 和 Skill 的区分是：Memory 是运行时状态（高频变化、任务特定），Skill 是可复用经验（低频变化、跨任务通用）。

核心数据文件：

文件	类型	写入规则	内容
task_state.json	结构化JSON	可覆写	Feature 列表、状态、验证记录
progress.md	追加式Markdown	append-only	每会话的进度摘要（完成了什么，完成率 %）
decisions.md	追加式Markdown	append-only	架构决策、环境约束、重规划事件
facts.md	追加式Markdown	append-only	智能体发现的非显而易见行为（[DISCOVERY]）、Verifier 记录的 API 契约偏差（[CONTRACT]）
handoffs/*.md	每会话生成	新文件	结构化交接文档

追加式治理。 Memory markdown 文件（progress/decisions/facts）强制 append-only——write 工具拒绝以 overwrite 模式写入这些文件。如果某条记录后来被发现是错的，不能删除，只能追加一条 [Correction] 条目。这确保记忆文件保持审计追踪的完整性。

去重与维护。 MemorySkillManager 在追加前检查相似条目（基于 MD5 哈希的滑动窗口去重）。定期维护操作（每 10 个会话）包括：去重 facts.md 中重复记录、检查疑似失效的 Skill。

信息分类原则。 不是所有信息都应持久化。系统的分类策略是：

- **写入持久化文件：** 非显而易见的 API 行为、架构决策、修复模式、验证结果
- **现场重新读取：** 文件内容（通过 read 工具按需获取，而非在交接中复制整个文件）

错误记录与记录污染。 系统的 append-only 策略天然防止了意外覆盖，但也可能积累错误记录。系统通过以下机制缓解：（1）confidence 标签（verified > observed > assumed > corrected > deprecated）标记记录的可信度；（2）[Correction] 机制允许在不删除原文的情况下标记修正；（3）定期维护进行去重。

四、实验分析

4.1 实验任务与设置

选用 NL2RepoBench 基准测试集中的 bleach 任务作为评测载体。Bleach 是一个 HTML 清理库，目标测试用例总数为 448 个。任务规范 start.md 描述了六个核心模块的 API 签名和行为要求：HTML 清理器（clean/Cleaner）、链接自动识别（linkify/Linker）、CSS 清理器（CSSSanitizer）、回调系统（nofollow/target_blank）、html5lib 适配层（Filter 基类和实体转换）、以及 URL 解析工具（urlparse）。任务规模明显超过单个上下文窗口——即使在 1M tokens 的上下文容量下，仅测试文件就包含数百条参数化用例，加上多轮编码和调试的交互历史，单会话无法完成全部工作。

实验设置两组对照运行：

- **实验组（有验证）：** 使用 run_bench_task.sh bleach 启动，验证功能全部开启（Tier-1 自动化测试 + Tier-2 采样审查 + 回归检测）。运行目录为 bench-bleach/。

- **对照组（无验证）**：使用 `run_bench_task_no_verify.sh bleach` 启动，通过 `verification.enabled: false` 关闭全部验证环节，模型自报的完成状态被直接信任。运行目录为 `bench-bleach-no-verify/`。

两组使用相同的 LLM 配置（DeepSeek V4 Pro，thinking 模式）和相同的会话预算（200 turns/会话，750K tokens/会话，共 40 会话上限，8 小时上限）。

4.2 实验组：有验证模式下的运行分析

4.2.1 总体运行情况

实验组共运行 5 个会话，总耗时 1.02 小时。Planner 将任务分解为 7 个里程碑、16 个 Feature，最终 14 个 Feature 状态为通过、1 个为进行中（F6.1）、1 个为待处理（F7.1），Feature 级完成率为 87.5%。任务因“停滞”（stalled）而非正常完成结束。

各会话运行概况如下：

会话	轮数	停止原因	耗时	说明
Session 1	47	budget_exhausted	447s	创建项目骨架，完成 F1.1、F1.2；检出反作弊违规
Session 2	37	budget_exhausted	252s	实现 HTML 清理链和高级特性，多个 Feature 通过验证
Session 3	46	budget_exhausted	304s	实现 Linkifier 和 CSS 清理器；F3.1 被 Tier-2 驳回
Session 4	47	budget_exhausted	356s	重写 F3.1（第 4 次尝试），实现回调系统和部分集成
Session 5	43	budget_exhausted	1086s	实现集成 Feature；F6.1 启动但未完成，会话因 token 耗尽结束

所有 5 个会话均因 `budget_exhausted`（token 预算耗尽）而结束，说明任务规模确实超出单会话容量，跨会话交接机制在整个运行过程中被频繁使用。

4.2.2 上下文管理效果

5 个会话共发生 5 次会话切换（4 次因预算耗尽、1 次因停滞），每次切换均生成了结构化的交接文档（`handoffs/session_NNNN.md`）。从运行轨迹来看，Bootstrap 序列使每个新会话能够快速恢复上下文——例如 Session 3 启动时读取了 Session 2 的交接文档和 F3.1 的失败记录，从而针对性地重写了 linkify 实现。

但在 Session 5 中暴露出当前上下文管理的一个缺陷：F6.1（`html5lib_shim` 模块）被标记为 `in_progress` 后，会话因 token 耗尽而结束。下一轮循环中，`pick_next_feature` 函数不将 `in_progress` 状态纳入选择范围（仅选取 FAILED/PENDING/BLOCKED），导致 F6.1 永远无法被重新调度。同时 F7.1（最终验证）依赖 F6.1，依赖未满足也无法被选中。系统判定无可用的 Feature 后触发“停滞”事件并终止。这是 `pick_next_feature` 算法的一个设计缺陷：未处理会话中断时遗留的 `in_progress` 状态。

4.2.3 验证机制表现

验证系统在本次实验中执行了关键的质量把关作用：

- **Tier-1 自动化验收**：共验证 15 个 Feature，所有 15 个均通过 Tier-1（pytest 退出码检查），说明模型至少保证了基本的测试通过。
- **Tier-2 独立审查**：对其中 5 个 Feature 启动了 Tier-2 LLM 审查（采样率约 33%），其中 4 个通过、1 个被驳回。唯一的驳回案例是 **F3.1 (linkify 核心函数)**：Tier-1 测试全部通过，但 Verifier 在审查中发现该实现对外部 URL 库的依赖关系与规范不符，判定为失败。这一案例恰好展示了 Tier-2 不可替代的价值：自动化测试只能验证"输出对输入是否正确"，无法验证"实现方式是否遵循了规范中的设计意图"。
- **回归检测**：在本次运行中未触发任何回归（新 Feature 未破坏已通过的 Feature）。
- **最终验证**：未执行，因为任务在 87.5% 完成度时停滞，`final_verification` 命令未被触发。

4.2.5 最终评估结果

使用 `eval.sh` 在独立的 Docker 测试镜像上运行完整测试套件，结果为：

- **测试收集**：203 items + 1 error（`test_clean.py` 因 ImportError 无法收集，说明 clean 模块的实现存在导入级错误）
- **通过/失败**：102 passed / 101 failed / 1 error
- **得分**：22.8% (102/448)
- **状态**：error（测试执行异常）

从测试文件维度的通过情况来看，`test_callbacks.py` (10/10 全部通过)、`test_parse_shim.py` (21/21 全部通过) 和 `test_unicode.py` (9/9 全部通过) 表现优秀，但 `test_css.py`（全部 33 个用例失败）和 `test_linkify.py`（大量失败）拖累了整体得分。`test_clean.py` 的 ImportError 导致整个测试文件被跳过，该错误在模型工作时未被发现——Agent Loop 中运行的是 pytest 的子集测试而非完整套件，这暴露了 Feature 级 acceptance 命令与全局测试套件之间的覆盖缺口。

4.3 对照组：关闭验证模式下的运行分析

4.3.1 总体运行情况

对照组共运行 4 个会话，总耗时 0.54 小时（约为实验组的 53%）。Planner 将任务分解为 6 个里程碑、12 个 Feature（与实验组的 16 个 Feature 不同，Planner 的分解具有非确定性）。模型自报全部 12 个 Feature 通过，Feature 级完成率为 100%。任务以"完成"（complete）状态正常结束。

各会话运行概况：

会话	轮数	停止原因	耗时	说明
Session 1	35	budget_exhausted	512s	基础骨架和导入
Session 2	36	budget_exhausted	546s	核心功能实现
Session 3	33	budget_exhausted	388s	高级特性实现
Session 4	35	idle_no_work	434s	模型自报全部完成，无更多工作

Session 4 的停止原因是 `idle_no_work` ——模型认为自己已经完成了所有工作，主动停止。

`pick_next_feature` 返回 None，`completion_ratio = 1.0`，最终验证因 `verification.enabled = false` 被自动标记为通过，任务正常结束。

4.3.2 与实验组的对比

维度	实验组（有验证）	对照组（无验证）
总会话数	5	4
总耗时	1.02 小时	0.54 小时
Planner 分解	16 Features	12 Features
Feature 完成率	87.5%（15/16 被验证，14 通过）	100%（12/12 自报通过）
结束原因	stalled（F6.1 卡在 in_progress）	complete（模型自报全部完成）
验证次数	15（Tier-1）+ 5（Tier-2）	0
Tier-2 驳回	1（F3.1）	N/A

从运行效率来看，对照组更快完成（0.54h vs 1.02h），会话数更少（4 vs 5）。这符合预期，关闭验证省去了 Tier-1 执行（pytest 运行）、Tier-2 LLM 审查会话、回归重跑的全部开销。

4.3.3 最终评估结果与"假完成"分析

尽管模型自报了 100% Feature 完成率和任务正常结束，最终评估结果揭示了问题的严重程度：

- **测试收集**：446 items（几乎覆盖了全部目标用例，远高于实验组的 203 items）
- **通过/失败**：179 passed / 263 failed
- **得分**：40.0%（179/448）

得分（40.0%）实际上高于实验组（22.8%），但这并不能简单解读为"关闭验证更好"。需要仔细分析：

对照组得分更高的原因：（1）对照组的模型将全部 token 用于编码，实现了更多的功能（446 个测试用例可收集 vs 实验组的 203 个），测试覆盖面积更大；（2）实验组因 F6.1 停滞而提前终止，失去了在后续会话中修复 `test_css.py` 和 `test_linkify.py` 中大量失败用例的机会；（3）实验组的 Planner 分解出了 16 个 Feature，每个 Feature 的工作量更小但切换开销更大。

"假完成"现象：对照组中，模型在 Session 4 主动停止并声称"所有工作已完成"，但实际测试通过率仅有 40.0%。从运行轨迹可以确认，这是典型的"模型幻觉完成"，模型在没有外部验证的情况下高估了自己的完成质量。具体表现包括：（1）`test_css.py` 全部 33 个用例失败，说明 CSS 清理器模块几乎完全没有正确实现；（2）`test_linkify.py` 中 263 个失败主要集中于链接识别的边界条件（括号匹配、端口解析、Unicode 处理）；（3）`test_html5lib_shim.py` 存在明显的 API 契约错误。这些缺陷如果经过 Tier-2 审查，很可能被及时发现并要求修正。

实验组停滞的影响：实验组仅得 22.8% 并非因为验证机制降低了代码质量，而是因为 F6.1 停滞导致模型失去了修复 `test_css.py` 和 `test_linkify.py` 的机会。如果修复 `pick_next_feature` 的 `in_progress` 处理缺陷，实验组应能在后续会话中显著提升得分。事实上，实验组在已覆盖的 203 个测试用例中的通过率为 50.2%（102/203），高于对照组的 40.1%（179/446）——验证机制帮助模型在已完成的 Feature 上达到了更高的质量。

4.3.4 Skill 与 Memory 使用情况

Skill 提取。实验组的 `fix-f3.1` Skill 被成功自动提取。F3.1（linkify 核心函数）经过 4 次尝试才最终通过验证，前 3 次失败的原因包括 API 签名与规范不一致、默认参数未使用 `DEFAULT_CALLBACKS`、以及 `__init__.py` 和子模块之间存在导入路径不统一的问题。编排器在 F3.1 最终通过后，调用 Planner/Flash LLM 分析了 F3.1 的会话转录，自动生成了 `fix-f3.1/SKILL.md`，其中记录了问题根因

(两个模块位置需要一致的函数签名和默认参数)、修复方法(以子模块为真相源, 顶层仅做薄封装重新导出)、以及关键教训(当函数需出现在两个模块位置时, 保持签名默认值同步的策略)。该 Skill 被标记为 `llm-extracted`、标签为 `session-3, F3`, 随后被重建的 `INDEX.md` 索引收录, 可供后续会话在 Bootstrap 阶段按需加载。

对照组(无验证)未产生任何 Skill 文件。这是因为验证被关闭后, 模型的任何实现都不会被 Tier-2 驳回或要求重试, Feature 一旦被模型标记为 `PASSING` 即被接受, 不存在"多次失败后最终成功"的触发场景。

Memory 文件。 两组运行的 Memory 文件均正常运作。`progress.md` 由编排器在每会话结束后自动追加进度摘要; `decisions.md` 记录了实验组 Session 1 的反作弊事件(`inspect.getsource` 两次调用被标记为可疑)以及架构决策; `facts.md` 在两个运行中均有内容, 但写入方式不同。实验组由 Verifier 写入了两条 `[CONTRACT]` 记录, 均为 F3.1 的 Tier-2 审查时发现的 API 契约偏差: (1) `bleach.linkify()` 的 `callbacks` 参数默认值应为 `DEFAULT_CALLBACKS` 而非 `None`; (2) `bleach.linkifier` 模块缺少独立的 `linkify` 函数。对照组(无验证)由编码智能体主动写入了两条 `[DISCOVERY]` 记录, 一条关于 `Linker._linkify_html` 中 `skip_tags` 参数未被实际使用的行为发现, 另一条关于 `Cleaner` 过滤器链的执行顺序(最外层先运行)与直觉相反的设计发现。这说明即使没有验证器, 编码智能体在遇到非显而易见的行为时仍会主动记录发现; 但对照组缺少验证器写入的 `[CONTRACT]` 条目——这类 API 契约偏差记录需要验证器在对照规范审查代码时才能产生, 编码智能体自身难以在实现过程中意识到这些偏差。交接文档(`handoffs/session_NNNN.md`)在两组中均由编排器自动生成, 为跨会话上下文传递提供了结构化的 git commit 信息、未解决问题列表和下一步行动建议。

4.4 关键发现与归因

发现一: 模型存在显著的"假完成"倾向。 对照组的运行轨迹提供了一手证据, 模型在 4 个会话后主动宣告任务完成, 但实际通过率仅 40.0%。这验证了本系统设计中"模型不能自行标记 Feature 为完成"这一核心约束的必要性。在实验组中, 同样是这个模型, 同样有"假完成"的倾向(F3.1 就是一个例子, 模型声称完成但 Tier-2 驳回), 但验证机制成功地拦截了这些虚假声明。

发现二: `pick_next_feature` 算法存在状态覆盖盲区。 实验组因 F6.1 遗留 `in_progress` 状态而提前终止, 这是 `pick_next_feature` 仅覆盖 `FAILED/PENDING/BLOCKED` 三种状态而遗漏 `IN_PROGRESS` 导致的。该发现直接促成了后续的代码修复: 将 `IN_PROGRESS` 加入选择范围, 优先级置于 `FAILED` 之前。

发现三: Tier-2 审查发现了 Tier-1 无法覆盖的缺陷。 F3.1 的驳回案例证明, 即使所有 `pytest` 测试通过, 代码仍可能存在设计层面的问题(如对外部库的不当依赖)。Tier-2 作为独立的"盲审"机制, 不受编码历史的偏见影响, 是验证体系中不可替代的一环。但 Tier-2 的代价也是真实的, 每次审查消耗额外的 LLM 调用, 延长了运行时间。

发现四: Feature 级验收与全局测试套件之间存在覆盖缺口。 实验组中 `test_clean.py` 的 `ImportError` 在所有 Feature 的 `acceptance` 命令中均未被发现, 因为每个 Feature 的 `pytest` 命令仅运行其相关的测试子集(通过 `-k` 过滤)。模型在会话中始终运行的是局部测试, 从未执行完整的测试套件, 因此这个导入级错误一直隐藏到最终评估。这提示需要在验证流程中增加阶段性的全局冒烟测试。

发现五: 验证的系统性代价需要权衡。 实验组比对照组多用了 89% 的时间 (1.02h vs 0.54h), 但 Feature 级质量更高(在已覆盖范围内 50.2% vs 40.1% 的通过率)。这种权衡在当前规模的任务上是可以接受的: 验证避免了模型在错误方向上持续投入, 而验证本身的开销(主要是 Tier-2 的 LLM 调用)占总运行时间的比例不高。但如果任务规模进一步扩大, Tier-2 的 30% 采样率可能带来显著的成本压力。

五、AI 工具在本项目中的使用情况

本项目在开发过程中使用 Claude Code 作为辅助开发工具。具体使用方式包括:

- **架构设计与迭代**：在确定三层架构（Orchestrator → Agent Loop → Verifier）后，与 AI 讨论各模块的职责边界、数据流和接口设计。AI 参与了 `task_state.json` 的状态机设计、会话切换策略的讨论、以及交接文档格式的定义。
- **核心代码编写**：`harness/` 目录下的所有 Python 模块（`orchestrator.py`、`agent_loop.py`、`verifier.py`、`task_state.py`、`context_manager.py`、`memory_skill.py`、`tools.py`、`llm_client.py`、`config.py`、`cli.py`）均由 AI 辅助编写。开发者提供设计意图和约束，AI 生成初始实现，然后通过人工审查和运行调试迭代修正。
- **Prompt 工程**：16 个系统提示词文件（`prompts/*.md`）由 AI 辅助起草，覆盖 Planner、Verifier、Coding Agent、Bootstrap、Skill Extractor、Session Summarizer 等多种角色。提示词经过多轮实验调整，例如 Planner 提示词中特别强化了 acceptance 必须为对象而非字符串的约束（这是实际运行中发现的常见 Planner 输出错误）。
- **Bug 修复**：在调试过程中发现并修复了多个问题，包括：`_cheat_flags` 未初始化导致的 `AttributeError`、`_merge_plans` 中 `replan` 后 `attempts` 未重置、`last_verified_session` 被非验证操作污染等。